

巴别塔之歌： 自然语言到代码的翻译

以线性结构为例

回顾

回顾

从计概到数算

过渡期：

- ▶ 语言：C or C++
 - ▶ 语法：常见操作是否熟悉（输入输出/分支循环/函数调用）
 - ▶ 算法：分治、贪心、递推、回溯
 - ▶ 工具：LLM、DEBUG、IDE
-

回顾

本阶段学习地图

四个核心：

- ▶ 理解问题：将复杂行为拆分成最小动作
- ▶ 找到模型：利用学过的方法转化问题
- ▶ 理解实现：将伪代码翻译成代码
- ▶ 行为优化：设计、实现、测试的闭环

01 从说人话开始：结构分析与最小动作

01 从说人话开始：结构分析与最小动作

自然语言分析：原理拆分

写代码之前先把问题说清楚

问题想的很清楚、原理理解的也很透彻，可是实现起来却： ...

01 从说人话开始：结构分析与最小动作

自然语言分析：原理拆分

写代码之前先把问题说清楚

问题想的很清楚、原理理解的也很透彻，可是实现起来却： ...

忽视语法细节，困在输入输出处理

01 从说人话开始：结构分析与最小动作

自然语言分析：原理拆分

写代码之前先把问题说清楚

问题想的很清楚、原理理解的也很透彻，可是实现起来却： ...

忽视语法细节，困在输入输出处理

忽略边界条件，特殊样例考虑不周

01 从说人话开始：结构分析与最小动作

自然语言分析：原理拆分

写代码之前先把问题说清楚

问题想的很清楚、原理理解的也很透彻，可是实现起来却： ...

忽视语法细节，困在输入输出处理

忽略边界条件，特殊样例考虑不周

结构嵌套错误，不能按照预期运行

01 从说人话开始：结构分析与最小动作

自然语言分析：原理拆分

写代码之前先把问题说清楚

问题想的很清楚、原理理解的也很透彻，可是实现起来却： ...

忽视语法细节，困在输入输出处理

忽略边界条件，特殊样例考虑不周

结构嵌套错误，不能按照预期运行

设计思路有误，不可简单照搬模板

01 从说人话开始：结构分析与最小动作

自然语言分析：原理拆分

写代码之前先把问题说清楚

问题想的很清楚、原理理解的也很透彻，可是实现起来却： ...

忽视语法细节，困在输入输出处理

忽略边界条件，特殊样例考虑不周

结构嵌套错误，不能按照预期运行

设计思路有误，不可简单照搬模板

人脑更擅长处理自然语言而不是语法细节，直接写代码容易陷入语法错误和细节泥潭。而结构分析可以把复杂问题拆成简单步骤

01 从说人话开始：结构分析与最小动作

不求甚解：认识伪代码

问题：计算一个班级的平均分

自然语言描述：输入是一组学生成绩，处理是把所有成绩加起来再除以总人数，输出是平均分

01 从说人话开始：结构分析与最小动作

不求甚解：认识伪代码

问题： 计算一个班级的平均分

自然语言描述： 输入是一组学生成绩，处理是把所有成绩加起来再除以总人数，输出是平均分

初始化： 总分、每个学生的信息、学生总人数

01 从说人话开始：结构分析与最小动作

不求甚解：认识伪代码

问题： 计算一个班级的平均分

自然语言描述： 输入是一组学生成绩，处理是把所有成绩加起来再除以总人数，输出是平均分

初始化： 总分、每个学生的信息、学生总人数

处理输入： 输入格式、输入的数量、终止条件

01 从说人话开始：结构分析与最小动作

不求甚解：认识伪代码

问题： 计算一个班级的平均分

自然语言描述： 输入是一组学生成绩，处理是把所有成绩加起来再除以总人数，输出是平均分

初始化： 总分、每个学生的信息、学生总人数

处理输入： 输入格式、输入的数量、终止条件

维护变量： 输入是否合法、是否存在相互影响、更新公式

01 从说人话开始：结构分析与最小动作

不求甚解：认识伪代码

问题： 计算一个班级的平均分

自然语言描述： 输入是一组学生成绩，处理是把所有成绩加起来再除以总人数，输出是平均分

初始化： 总分、每个学生的信息、学生总人数

处理输入： 输入格式、输入的数量、终止条件

维护变量： 输入是否合法、是否存在相互影响、更新公式

总结输出： 计算输出、调整格式

01 从说人话开始：结构分析与最小动作

伪代码示例与练习

Algorithm 1 DM-SAC-H:

- 1: Initialize all the networks, $\varphi^\emptyset, \theta^\emptyset, \vartheta^\emptyset, \bar{\varphi}^\emptyset, \varphi^\partial, \theta^\partial, \vartheta^\partial, \bar{\varphi}^\partial$ and ℓ .
 - 2: **repeat:**
 - 3: **for** each time slot **do**
 - 4: Obtain $a(t), s(t+1), r(s(t), a(t))$ based on the policy ℓ and current state $s(t)$.
 - 5: Save the sample $\{s(t), a(t), s(t+1), r(s(t), a(t))\}$ to replay D .
 - 6: Generate samples for unavailable actions and save them to replay D .
 - 7: **end for**
 - 8: **for** each training epoch **do**
 - 9: Update $Q_{\varphi^\emptyset}$ based on (50), $Q_{\theta^\emptyset}$ based on (52), $Q_{\vartheta^\emptyset}$ based on (54) and samples from (59).
 - 10: Update Q_{φ^∂} based on (51), Q_{θ^∂} based on (53), Q_{ϑ^∂} based on (55) and samples from (59).
 - 11: Soft update $Q_{\bar{\varphi}^\emptyset}$ and $Q_{\bar{\varphi}^\partial}$.
 - 12: Update ℓ based on (56), (57) and (58).
 - 13: **end for**
 - 14: **until** convergence.
-

01 从说人话开始：结构分析与最小动作

化整为零：拆解最小动作

最小动作：不需要再继续拆分的基本操作，相对于问题而定义

- ▶ 链表：创建节点、删除节点、寻找下一个节点、取节点值
- ▶ 栈/队列：出、入、观察顶层元素
- ▶ 字符串：字符位比较、拼接、查找

01 从说人话开始：结构分析与最小动作

最小动作集：定义节点

```
struct Node {  
    int val;  
    Node* next;  
};  
int main() {  
    Node* head = NULL;  
    ...  
}
```

```
int main() {  
    struct Node {  
        int val;  
        Node* next;  
    };  
    Node* head = NULL;  
    ...  
}
```

```
struct Node {  
    int val;  
    struct Node* next;  
};  
int main() {  
    struct Node* head = NULL;  
    ...  
}
```

```
typedef struct Node {  
    int val;  
    struct Node* next;  
} Node;  
int main() {  
    Node* head = NULL;  
    ...  
}
```

01 从说人话开始：结构分析与最小动作

最小动作集：创建节点

```
struct Node {  
    int val;  
    Node* next;  
};  
int main() {  
    Node* head = NULL;  
    ...  
}
```

```
int main() {  
    struct Node {  
        int val;  
        Node* next;  
    };  
    Node* head = NULL;  
    ...  
}
```

```
struct Node {  
    int val;  
    struct Node* next;  
};  
int main() {  
    struct Node* head = NULL;  
    ...  
}
```

```
typedef struct Node {  
    int val;  
    struct Node* next;  
} Node;  
int main() {  
    Node* head = NULL;  
    ...  
}
```

01 从说人话开始：结构分析与最小动作

最小动作集：创建节点链表

```
Node* head = new Node;  
head->val = 1;  
head->next = nullptr;  
  
Node* cur = head;  
  
Node* newNode = new Node;  
newNode->val = 2;  
newNode->next = nullptr;  
  
cur->next = newNode;  
cur = newNode;    // 向后移动
```


01 从说人话开始：结构分析与最小动作

最小动作集：创建节点链表

```
int n = 5;                // 节点数量
Node* head = new Node;    // 创建头节点
head->val = 1;
head->next = nullptr;

Node* cur = head;         // cur 指向当前尾节点

for (int i = 2; i <= n; i++) {
    Node* newNode = new Node; // 创建新节点
    newNode->val = i;
    newNode->next = nullptr;

    cur->next = newNode;      // 连接到链表
    cur = newNode;          // 移动尾指针
}
```

```
int n = 5;
Node* head = new Node;
head->val = 1;
head->next = nullptr;

Node* cur = head;
int i = 2;

while (i <= n) {
    Node* newNode = new Node;
    newNode->val = i;
    newNode->next = nullptr;

    cur->next = newNode;
    cur = newNode;

    i++;                // 计数器更新
}
```

01 从说人话开始：结构分析与最小动作

最小动作集：创建节点链表

```
// 递归函数：创建从 start 到 n 的链表
Node* createList(int start, int n) {
    if (start > n) {
        return nullptr;          // 终止条件（循环机制是否可以更换？）
    }
    Node* node = new Node;        // 创建节点
    node->val = start;             // 赋值
    node->next = createList(start + 1, n); // 递归创建下一个节点

    return node;                 // 返回当前节点
}

int main() {
    int n = 5;
    Node* head = createList(1, n);
    return 0;
}
```

01 从说人话开始：结构分析与最小动作

最小动作集：在链表中移动

```
Node* cur = head;           // 指向头节点
while (cur != nullptr) {    // 检查是否到达链表末尾
    std::cout << cur->val << " "; // 访问节点数据
    cur = cur->next;         // 移动到下一个节点
}
```

```
for (Node* cur = head; cur != nullptr; cur = cur->next) {
    std::cout << cur->val << " ";
}
```


01 从说人话开始：结构分析与最小动作

最小动作集：链表新增节点

```
//头部新增
Node* newNode = new Node;
newNode->val = 5;
newNode->next = head;    // 新节点 next 指向原头节点
head = newNode;         // 更新头指针
```

```
//中部新增
Node* newNode = new Node;
newNode->val = 15;

Node* cur = head;
for (int i = 1; i < k - 1; i++) {
    cur = cur->next;    // 移动到前驱节点
}

newNode->next = cur->next; // 新节点指向后继节点
cur->next = newNode;      // 前驱节点指向新节点
```

```
//尾部新增
Node* newNode = new Node; // 创建节点
newNode->val = 10;         // 赋值
newNode->next = nullptr;   // 初始化 next

Node* cur = head;
for (int i = 1; i < n ; i++) {
    cur = cur->next;    // 移动到前驱节点
}
cur->next = newNode;     // 前驱节点 next 指向新节点
cur = newNode;          // 可选：移动尾指针
```

01 从说人话开始：结构分析与最小动作

最小动作集：链表删除节点

```
//头部删除
Node* temp = head;    // 保存头节点指针
head = head->next;    // 更新头指针
delete temp;          // 释放原头节点
```

```
//中部删除
Node* cur = head;
for (int i = 1; i < k - 1; i++) {
    cur = cur->next;    // 移动到前驱节点
}

Node* temp = cur->next; // 保存要删除的节点
cur->next = temp->next;  // 前驱节点指向后继节点
delete temp;            // 释放内存
```

```
//尾部删除
Node* cur = head;
while (cur->next->next != nullptr) {
    cur = cur->next;    // 移动到倒数第二个节点
}

Node* temp = cur->next; // 保存尾节点
cur->next = nullptr;    // 前驱 next 置空
delete temp;            // 释放尾节点
```

01 从说人话开始：结构分析与最小动作

随堂练习：链表反转

```
struct Node {  
    int val;  
    Node* next;  
};  
  
Node* reverseList(Node* head) {  
    Node* prev = /*_____*/;    // 初始化前驱节点  
    Node* curr = /*_____*/;    // 初始化当前节点  
    while (curr != nullptr) {  
        Node* nextTemp = /*_____*/;    // 保存下一个节点  
        curr->next = /*_____*/;    // 反转指针  
        prev = /*_____*/;    // 前驱节点向前移动  
        curr = /*_____*/;    // 当前节点向前移动  
    }  
    return /*_____*/;    // 返回新头节点  
}  
  
int main() {  
    ...  
}
```

02 转译的损耗：常见理解歧义

02 转译的损耗：常见理解歧义

天平倾斜：如何理解等于号

这里的 = 是“赋值”还是“比较”？ 它发生了什么？

```
prev = curr;  
curr = nextTemp;
```


02 转译的损耗：常见理解歧义

天平倾斜：如何理解等于号

```
prev = curr;  
curr = nextTemp;
```

- ▶ `prev = curr;`
- ▶ 人脑直观理解：把 `curr` 的值给 `prev`
- ▶ 计算机层面：`prev` 的内存地址保存 `curr` 指向的节点地址
- ▶ 关键点：这是指针赋值，不复制节点内容，只复制地址
- ▶ `curr = nextTemp;`
- ▶ 不是“`curr` 变成了下一个节点”，而是指针移动

最小动作拆解 1. 找到左边变量的存储位置 2. 计算右边表达式的值 3. 写入左边位置

02 转译的损耗：常见理解歧义

天平倾斜：如何理解等于号

```
prev = curr;  
curr = nextTemp;
```

最小动作拆解

- ▶ 1. 找到左边变量的存储位置
- ▶ 2. 计算右边表达式的值
- ▶ 3. 写入左边位置
- ▶ 关键点：房子模型

02 转译的损耗：常见理解歧义

天平倾斜：如何理解等于号

```
prev = curr;  
curr = nextTemp;
```

- ▶ 1. `prev = curr;` 会复制整个节点？ → 实际只是指针复制
- ▶ 2. `curr = nextTemp;` 改变了原链表？ → 实际只是移动了本地指针
- ▶ 3. 把 `=` 理解成数学等号？ → 并非“相等判断”，而是“赋值操作”

02 转译的损耗：常见理解歧义

流式执行：阻塞式赋值的陷阱

可以更改这几行的顺序吗？

```
Node* nextTemp = curr->next;  
curr->next = prev;  
prev = curr;  
curr = nextTemp;
```

02 转译的损耗：常见理解歧义

流式执行：阻塞式赋值的陷阱

```
Node* nextTemp = curr->next;  
curr->next = prev;  
prev = curr;  
curr = nextTemp;
```

- ▶ 1. 必须先保存下一节点指针 → `nextTemp = curr->next`，否则信息丢失
- ▶ 2. 指针反转 `curr->next = prev` → 顺序敏感
- ▶ 3. `prev = curr`; 移动前驱指针，依赖 `curr` 的当前值
- ▶ 4. `curr = nextTemp`; 移动当前指针，必须在前面步骤之后执行
- ▶ ⚠ 如果改变顺序 → 链表破坏、指针指向错误

02 转译的损耗：常见理解歧义

流式执行：阻塞式赋值的陷阱

阻塞式赋值

- ▶ 代码必须按照特定顺序执行，每一步都依赖前一步的结果
- ▶ 执行顺序按照代码书写的顺序进行，不能随意调整
- ▶ 如果中途有函数调用关系，则优先执行函数内部的代码，完成后再继续当前函数的下一行
- ▶ 在进入循环/递归时，必须保证每次迭代/调用的状态正确，不能因为顺序错误导致状态混乱

02 转译的损耗：常见理解歧义

拓展：非阻塞式赋值

```
// Verilog 非阻塞赋值示例
always @(posedge clk) begin
    prev <= curr;
    curr <= nextTemp;
end
```

- ▶ 1. 非阻塞赋值 ($\leq$)：所有右值同时读取，左值同时更新
- ▶ 2. 顺序不会阻塞，可以并行理解多个赋值
- ▶ 3. 与阻塞赋值 ($=$) 对比：阻塞赋值必须严格顺序执行
- ▶ 4. 可以理解为“先看右边值，再同时更新所有左边”

02 转译的损耗：常见理解歧义

流式执行：阻塞式赋值的陷阱

阻塞式赋值

- ▶ 代码必须按照特定顺序执行，每一步都依赖前一步的结果
- ▶ 执行顺序按照代码书写的顺序进行，不能随意调整
- ▶ 如果中途有函数调用关系，则优先执行函数内部的代码，完成后再继续当前函数的下一行
- ▶ 在进入循环/递归时，必须保证每次迭代/调用的状态正确，不能因为顺序错误导致状态混乱

02 转译的损耗：常见理解歧义

循环新生：迭代赋值分析

正确循环的条件

- ▶ a. 初始化变量状态正确
- ▶ b. 每次迭代右值使用前已经保存
- ▶ c. 左值更新后能满足下一次迭代的需要
- ▶ d. 循环终止条件清晰且可达

02 转译的损耗：常见理解歧义

循环新生：迭代赋值分析

泛化理解循环设计：只要连续两次迭代之间变量衔接正确，整个循环就正确。

- ▶ 1. 循环本质 → 重复执行一组动作，每次迭代处理局部最小动作
- ▶ 2. 变量衔接原则 → 检查每次迭代结束时变量状态能否满足下一次迭代的初始要求
- ▶ 3. 抽象方法 → 可以连续观察两次迭代，验证左值更新、右值读取、依赖顺序是否正确

02 转译的损耗：常见理解歧义

例题：多项式相加（链表）

将两个无序多项式相加，要求合并同类项并按幂次降序输出。

```
// 输入形式（示例）
2 20 -1 17 5 9 -7 7 16 5 10 4 22 2 -15 0 [结束标志]
2 19 7 7 3 17 4 4 15 10 -10 5 13 2 -7 0 [结束标志]
// 输出形式
[a b]的序列 表示  $a * x^b + \dots$ 
```

- ▶ 核心问题：“逐项处理 + 排序 + 合并同类项”
- ▶ 每次循环只做“最小动作”：处理一个项，并保证结果可被下一次继续使用

02 转译的损耗：常见理解歧义

例题：多项式相加（链表）

▷ 循环设计思路

▷ a. 逐对读取（系数, 幂次）

▷ b. 将同幂次项累加

▷ c. 统一排序

▷ d. 最终输出

02 转译的损耗：常见理解歧义

例题：多项式相加（链表）

- ▶ 关键循环拆解
 - ▶ 循环1：读取输入（构建结构）
 - ▶ 循环2：合并同类项（累加）
 - ▶ 循环3：按幂排序输出
- ▶ 迭代衔接分析
 - ▶ a. 每次读取后，当前项必须正确加入“已有结果”
 - ▶ b. 累加结果必须能被下一次迭代继续使用
 - ▶ c. 任意两次迭代之间，状态（当前结果集合）必须一致

02 转译的损耗：常见理解歧义

代码介绍

03 标准模板库：使用与等效

03 标准模板库：使用与等效

标准模板库：使用与等效

从 map 出发，理解 STL 的本质：用现成结构替代手写循环与状态维护。

```
map poly;  
poly[exp] += coef;
```

▶ map 做了什么 →

- ▶ a. 自动查找“是否已有该幂次”
- ▶ b. 自动维护有序性（按 key 排序）
- ▶ c. 自动完成插入 / 更新

03 标准模板库：使用与等效

标准模板库：使用与等效

- ▶ 等效写法（链表版） →
 - ▶ a. 遍历链表找位置
 - ▶ b. 判断是否合并
 - ▶ c. 手动插入节点
- ▶ 核心对比 →
 - ▶ map = “封装好的循环 + 状态结构”
 - ▶ 链表 = “手写循环 + 手动维护状态”

03 标准模板库：使用与等效

标准模板库：使用与等效

- ▶ STL 的本质是“常见模式的抽象”
- ▶ 帮你保证每次操作后状态始终正确
- ▶ 使用原则：
 - ▶ 能用 STL → 优先用（减少错误）
 - ▶ 需要控制细节 → 自己实现（链表/数组）

03 标准模板库：使用与等效

常见 STL 结构：适用场景与核心操作

不同 STL 容器，本质是不同“状态维护方式”的封装。

```
// 常见 STL 示例
vector v;
map m;
unordered_map um;
set s;
queue q;
stack st;
```


03 标准模板库：使用与等效

常见 STL 结构：适用场景与核心操作

```
// 常见 STL 示例  
vector v;  
map m;  
unordered_map um;  
set s;  
queue q;  
stack st;
```

▶ 1. vector（动态数组） →

▶ 适用：顺序存储、频繁访问

▶ 核心操作：push_back(), pop_back(), size(), []

03 标准模板库：使用与等效

常见 STL 结构：适用场景与核心操作

▶ 2.map（有序键值对） →

▶ 适用：需要排序 + 查找 + 统计

▶ 核心操作：m[key], insert(), find(), erase()

▶ 3.unordered_map（哈希表） →

▶ 适用：快速查找（不关心顺序）

▶ 核心操作：um[key], insert(), find(), erase()

02 转译的损耗：常见理解歧义

常见 STL 结构：适用场景与核心操作

▶ 4. stack（栈） →

▶ 适用：后进先出（递归、括号匹配）

▶ 核心操作：push(), pop(), top()

▶ 5. queue（队列） →

▶ 适用：先进先出（层序遍历、调度）

▶ 核心操作：push(), pop(), front()

03 标准模板库：使用与等效

常见 STL 结构：适用场景与核心操作

- ▶ 6.set（有序去重） →
 - ▶ 适用：去重 + 自动排序
 - ▶ 核心操作：insert(), erase(), find()
- ▶ vector → 顺序状态
- ▶ map / set → 有序状态
- ▶ unordered_map → 哈希状态
- ▶ stack / queue → 受限访问状态

04 例题选讲：

从算法原理开始

04 例题选讲：

子串查找： 快慢指针

描述

你设计了一个新的加密技术，可以用一种聪明的方式在一个字符串的字符间插入随机的字符串从而对信息进行编码。由于专利问题，我们将不会详细讨论如何在原有信息中产生和插入字符串。不过，为了验证你的方法，有必要写一个程序来验证原来的信息是否全在最后的字符串之中。

给定两个字符串s和t，你需要判断s是否是t的“子列”。也就是说，如果你去掉t中的某些字符，剩下字符将连接而成为s。

输入

输入包括多个测试样例。每一个都是由空格分隔的由字母数字ASCII字符组成的两个特定的字符串s和t。s和t的长度不超过100000。

输出

对于每个测试样例，如果s是t的“子列”，则输出“Yes”，否则输出“No”

样例输入

```
sequence subsequence
person compression
VERDI vivaVittorioEmanueleReDiItalia
caseDoesMatter CaseDoesMatter
```

04 例题选讲:

子串查找：快慢指针

```
while (i < s.size() && j < t.size()) {  
    if (s[i] == t[j]) {  
        i++; // 匹配成功, 移动 s 指针  
    }  
    j++; // t 总是向前  
}  
  
if (i == s.size()) cout << "Yes" << endl;  
else cout << "No" << endl;
```

04 例题选讲：

滑动窗口： 单调队列

描述

给定一个长度为n ($n \leq 10^6$) 的数组。有一个大小为k的滑动窗口从数组的最左端移动到最右端。你可以看到窗口中的k个数字。窗口每次向右滑动一个数字的距离。
下面是一个例子：
数组是 [1 3 -1 -3 5 3 6 7], $k = 3$ 。

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

你的任务是得到滑动窗口在每个位置时的最大值和最小值。

04 例题选讲:

滑动窗口：单调队列

```
//此处展示降序优先队列，维护最大值
for (int i = 0; i < n; i++) {
    // 1. 移除窗口外的元素
    if (!dq.empty() && dq.front() <= i - k) {
        dq.pop_front();
    }

    // 2. 维护单调递减（小的全部弹掉）
    while (!dq.empty() && A[dq.back()] < A[i]) {
        dq.pop_back();
    }

    // 3. 加入当前元素
    dq.push_back(i);
    ...
}
```

04 例题选讲:

滑动窗口：单调队列

使用带头结点的单链表存储一个多项式，设计多项式加法、乘法及显示多项式的算法。

提示：

- 1) 使用带头结点单链表存储表示一个多项式，多项式结点按指数降序链接；
- 2) 结点可以如下定义：

```
struct Node{  
    int exp; //指数  
    int coef; //系数  
    struct Node *next;  
};
```

```
typedef Node* Poly;
```

- 2) 设计函数Poly add(const Poly & A,const Poly & B)完成两个多项式的加法运算；
- 3) 设计函数Poly mul(const Poly & A,const Poly & B)完成两个多项式的乘法运算；
- 4) 设计函数void printPolynomial(const Poly &)显示输出一个多项式。
- 5) 要求在main函数中进行测试。
- 6) 多项式 $3x^5-7x+6$ 用一个整数序列表示为：3 5 - 7 1 6 0 0 0，多项式项的指数 ≤ 100 。

04 例题选讲:

多项式运算与输出: 有序链表的维护

```
Poly createPolynomial() {
    Poly head = new Node{0, 0, nullptr}; // 头结点
    Node* tail = head;
    int coef, exp;
    while (cin >> coef >> exp) {
        if (coef == 0 && exp == 0) break;
        Node* p = new Node{exp, coef, nullptr};
        tail->next = p;
        tail = p;
    }
    return head;
}
```

```
void printPolynomial(const Poly &P) {
    Node* cur = P->next;
    bool first = true;
    while (cur) {
        if (cur->coef > 0 && !first) cout << "+";
        if (cur->exp == 0) cout << cur->coef;
        else if (cur->exp == 1) cout << cur->coef << "x";
        else cout << cur->coef << "x^" << cur->exp;
        first = false;
        cur = cur->next;
    }
    cout << endl;
}
```

04 例题选讲:

多项式运算与输出：有序链表的维护

```
Poly add(const Poly &A, const Poly &B) {
    Poly head = new Node{0, 0, nullptr};
    Node* tail = head;
    Node* p = A->next;
    Node* q = B->next;
    while (p && q) {
        if (p->exp > q->exp) {
            tail->next = new Node{p->exp, p->coef, nullptr};
            p = p->next;
        } else if (p->exp < q->exp) {
            tail->next = new Node{q->exp, q->coef, nullptr};
            q = q->next;
        } else { // 指数相同
            int sum = p->coef + q->coef;
            if (sum != 0) tail->next = new Node{p->exp, sum, nullptr};
            p = p->next;
            q = q->next;
        }
        if (tail->next) tail = tail->next;
    }
    while (p) { tail->next = new Node{p->exp, p->coef, nullptr}; tail = tail->next; p = p->next; }
    while (q) { tail->next = new Node{q->exp, q->coef, nullptr}; tail = tail->next; q = q->next; }
    return head;
}
```

04 例题选讲:

多项式运算与输出：有序链表的维护

```
Poly mul(const Poly &A, const Poly &B) {
    Poly result = new Node{0, 0, nullptr};
    for (Node* p = A->next; p; p = p->next) {
        for (Node* q = B->next; q; q = q->next) {
            int e = p->exp + q->exp;
            int c = p->coef * q->coef;
            // 插入到result中 (按指数降序)
            Node* prev = result;
            Node* cur = result->next;
            while (cur && cur->exp > e) { prev = cur; cur = cur->next; }
            if (cur && cur->exp == e) { // 已存在指数
                cur->coef += c;
                if (cur->coef == 0) { // 系数为0就删除节点
                    prev->next = cur->next;
                    delete cur;
                }
            } else { // 插入新节点
                Node* newNode = new Node{e, c, cur};
                prev->next = newNode;
            }
        }
    }
    return result;
}
```


04 例题选讲:

模式匹配: KMP

```
vector buildNext(const string &P) {  
    int m = P.size();  
    vector next(m, 0);  
    int j = 0; // 当前匹配长度  
  
    for (int i = 1; i < m; ++i) {  
        while (j > 0 && P[i] != P[j]) {  
            j = next[j-1]; // 回退状态  
        }  
        if (P[i] == P[j]) j++;  
        next[i] = j; // 更新状态  
    }  
    return next;  
}
```

04 例题选讲:

模式匹配: KMP

```
vector KMP(const string &T, const string &P) {  
    vector res;  
    vector next = buildNext(P);  
    int n = T.size(), m = P.size();  
    int j = 0; // 当前匹配长度  
  
    for (int i = 0; i < n; ++i) {  
        while (j > 0 && T[i] != P[j]) j = next[j-1];  
        if (T[i] == P[j]) j++;  
        if (j == m) {  
            res.push_back(i - m + 1); // 匹配成功  
            j = next[j-1]; // 状态衔接, 继续寻找下一个  
        }  
    }  
    return res;  
}
```

05 课后练习

05 课后练习：

前缀表达式、中缀表达式、后缀表达式的转换

课后说明

致谢

Thanks!

Mar.2026
